

Why I want Concepts, and why I want them sooner rather than later

Abstract

This paper provides highly-opinionated statements, anecdotes and personal opinions explaining why the author thinks Concepts should go into C++17 even if no Conceptified standard library parts are included in C++17.

Contents

- [In brief](#)
- [Concepts are superior to the work-around techniques we have to apply today](#)
- [Concepts allow constraining function arguments without turning off deduction and without disturbing arity](#)
- [Concepts make it much easier to write overloads with mutually exclusive constraints](#)
- [Concepts enable a broad range of constraint designs](#)
- [So why shouldn't we ship Concepts and a Conceptified standard library at the same time?](#)
- [But shouldn't the design of Concepts be validated by applying them to the library?](#)
- [Famous last words](#)

In brief

Concepts are ready to go in; furthermore they provide important facilities that are extremely burdensome and expert-only to mimic in C++14 and below, and I want to be able to begin applying those facilities in practice. I do not want to wait for a conceptified standard library before being able to do so. I have analyzed the applicability of Concepts in a limited fashion for the interfaces and the implementation of the standard library, and I do not believe applying Concepts in the standard library will result in design changes to Concepts.

Based on that confidence, I reiterate: it's time to ship Concepts the language facility, and we should not wait for the standard library to be partially or completely conceptified before doing so.

Concepts are superior to the work-around techniques we have to apply today

There's a whole host of things Concepts make much more direct and convenient to express than the work-arounds we use today. It's perhaps valuable to look at some such cases with examples, which I will do in the subsequent sections. Here's a teaser list:

- Concepts allow constraining function arguments without turning off deduction and without disturbing the meta-arity of a constrained function template.
- Concepts make it much easier to write overloads with mutually exclusive constraints.
- Concepts enable a broad range of constraint designs, giving the interface designer various tools to decide what sort of abstractions a particular interface needs.

Concepts allow constraining function arguments without turning off deduction and without disturbing arity

We start with something very simple:

```
template <class T> void f(T);
```

The problem here is that while this function template does deduction, it provides no actual interface. If we want to constrain its function parameter, we may initially try this:

```
template <class T> void f(enable_if_t<my_trait_v<T>, T>);
```

We just turned off deduction. That also means that the technique will not work for constructor templates. We may try a different approach:

```
template <class T> auto f(T) -> enable_if_t<my_trait_v<T>, void>;
```

Now we have a constraint for the parameter in the return type. Same applies to using a decltype of an expression in the trailing-return-type. This won't work with constructor templates either. We can resort to using constraints on the template argument:

```
template <class T, typename = enable_if_t<my_trait_v<T>>> void f(T);
```

Now we have changed the function template's meta-arity from one to two. All of these solutions are work-arounds. With concepts, we can do

```
void f(MyConcept);
```

The constraint for the parameter is at the parameter. The function template will still deduce the template argument. The arity of the function template is unchanged, as is its meta-arity.

Concepts make it much easier to write overloads with mutually exclusive constraints

It's often desirable to constrain overloads of function templates so that one overload is invoked when the template argument satisfies certain criteria, and another overload if not. Let's look at a thrilling example of what a person too clever for his own good will write utilizing "nifty" techniques in C++14:

```
template <class T> void f(T, decltype(declval<T>().f()))* = 0);
```

Pardon me. I should've warned you. The person is indeed too clever for his own good, and he is very dangerous when given "nifty" facilities. He didn't want to write a trait, because chances are that he's never going to reuse the constraint on the function, and is thus unwilling to separate it into a trait. So, he gets the declval of the template parameter, constructs the expression, and turns its type into a pointer (and completely fails to take into account references) because he wanted to support functions returning void, and defaults the argument in order to allow users to just call the function with one argument.

After all this, the "expression constraint" doesn't work with functions that return references, the whole constraint can be accidentally bypassed by invoking the function with two arguments, and no non-expert will have any hope of ever understanding what's going on there. To add insult to injury, this clever person can't figure out how to declare the overload that is supposed to be taken when the constraint is not satisfied. Any non-trait attempts he may make result in hard errors.

In contrast, with Concepts, here's what this too-clever programmer will write:

```
template <class T> void f(T) requires requires (T t) {t.f()};  
template <class T> void f(T);
```

The overloads are mutually exclusive, and everything is fine. If this programmer so chooses, he can write an actual Concept for the constraint, and apply separation of concerns that way. With the trait-based solutions, the programmer **has to write a trait**, for every expression he wants to constrain on, because otherwise he can't write the non-matching overload that requires a negation of the trait. He will also potentially run into the aforementioned problems with deduction, placement of the constraint, or arity. Sure, a requires-clause is not exactly next to the parameter either, but at least it's not hiding inside a trailing return type.

Concepts enable a broad range of constraint designs

The previous section very much hinted towards the topic of this section. For expression constraints, choosing an overload for which an expression is not valid requires writing a trait. It can't be written in an ad-hoc manner straight on the function template declaration. With Concepts, that is possible, and the code design evaluation path towards a real separate Concept is palatable, as is the evaluation path back towards an ad-hoc constraint. None of the issues with the C++14 techniques for constraints arise easily.

With Concepts, the programmer can decide what kind of an abstraction level is suitable for his constraints, and it's possible to mix and match Concepts and traits and requires-clauses fairly easily. None of that design leeway is attainable with the C++14 techniques.

So why shouldn't we ship Concepts and a Conceptified standard library at the same time?

I don't want to wait for the conceptification of the standard library before I can use the techniques illustrated in this paper (and there's more than the examples in this paper). There's a lot of code that can begin to benefit from those techniques even without a conceptified library. In many cases, such code wouldn't benefit much or at all from a conceptified library. Yes, I'm fully aware of the original goal of Concepts being to create something that can reasonably be used for the interfaces in the standard library; however, the language facilities it ended up providing are so alluring that they in somecases overshadow the benefits of some day having a constrained standard library.

I can start applying some of these techniques in some modern codebases now, in some modern codebases as soon as GCC 6 is out, and in some codebases when there's more than one compiler supporting Concepts. We currently have basic library Concepts in the Ranges TS, and any further conceptified library will likely just begin to form after we have gotten that TS to the publication phase. From that point, it takes some serious effort and time to further conceptify our standard library. It's not unreasonable to expect this process to take a couple of years, and all the while there would be a vast amount of code that could immensely benefit from Concepts the language feature.

But shouldn't the design of Concepts be validated by applying them to the library?

Some audiences think we cannot be sure that we got the design of Concepts right before we apply Concepts to the standard library. Furthermore, to some audiences this means just applying them into the specification in a somewhat abstract sense, and to some it means that Concepts need to be applied to the implementation as well. Even further, some audiences think Concepts should be applied to both the standard interfaces and the internal facilities of a library implementation. The claim is that such an exercise is bound to reveal bugs in the Concepts design.

Let's go from bottom to top for those concerns:

The applicability of Concepts to the internal facilities and the standard interfaces of a library implementation has been explored in a limited fashion. I have implemented certain internal facilities with fake Concepts, and tried out the techniques with actual Concepts. I didn't find any significant design changes I would want to have for Concepts. Extension ideas, yes; but such extensions are not really in scope for Concepts at this time - the extension ideas were mostly for ways to make implementing the guts of the library easier, and Concepts weren't designed for that; they were designed to be able to constrain the interfaces of the library, not for constraining the implementation.

There's been somewhat limited amounts of field experience with even the main use case of Concepts, constraining library interfaces. Even so, we have had some feedback from adventurous library writers, and thus far there have been fairly minor gripes about the Concepts design. They have been mostly requests to simplify Concepts (at the cost of decreased functionality, which was one of the reasons why those design issues were rejected by EWG) and requests to relax some limitations (which has been done to a certain extent). No really major design issues have been reported by anyone. That can certainly be an indication of small amounts of field experience, but I daresay that it's an indication that the design is sound.

We have had some amounts of specification experience with libraries, such as the Ranges TS. As far as I know, no major design issues for Concepts have come up. I very much doubt applying Concepts to more library facilities would reveal such issues, because I boldly claim that such issues aren't there.

Famous last words

I realize that the previous section might not be convincing to some people, and that shipping Concepts the language feature is a

leap of faith. That may well be so, but programmers are aching to get the language feature into their hands, and it's high time we ship it to them. Conceptifying the standard library will take its time, and we will not find major design issues for Concepts while doing it. We shouldn't keep programmers waiting for the language feature out of concern for hypothetical design issues that we have no proof of, have some amounts of counter-proof of, and very likely don't exist.

For the benefit of C++ users everywhere, let's ship Concepts the language feature in C++17.